

EVIDENCIA DE CODIFICACIÓN DE MÓDULOS

Tienda Los Boyacos — Plataforma de e-commerce

Aprendiz	Marino Pérez Marín
Programa	Análisis y Desarrollo de Software (ADSO) — SENA
Ficha	3070108
Proyecto	Tienda Los Boyacos
Repositorio	github.com/MARINOPEREZ/tiendalosboyacos-python (rama main)
Fecha	Septiembre de 2026

Este documento reúne la evidencia de codificación de los módulos del sistema siguiendo los requerimientos funcionales del proyecto: arquitectura y capas, diagramas (clases, paquetes, componentes), mapa de navegación, mecanismos de seguridad, librerías y frameworks por capa, patrones de diseño aplicados, pruebas unitarias, configuración de entornos y el detalle de los módulos codificados o corregidos durante esta actividad.

1. Identificación y alcance de la evidencia

Tienda Los Boyacos es una plataforma de e-commerce para productos artesanales del departamento de Boyacá, Colombia, desarrollada como proyecto académico del programa ADSO (ficha 3070108) y, a la vez, como aplicación funcional. El desarrollo se avaló explícitamente con **Python 3.13 + Flask** en el backend y **Jinja2 + JavaScript vanilla** en el frontend — sin React ni otro framework SPA — decisión tomada por el instructor del programa y vigente hasta la finalización de las actividades de aprendizaje.

Esta evidencia documenta el estado real del código verificado directamente sobre el sistema de archivos del proyecto (no solo sobre lo previamente documentado), y agrega dos piezas de codificación nuevas resueltas en esta actividad:

- Un **módulo administrativo completo de gestión de categorías** (alta, edición y baja), que antes no existía — solo existía la asociación producto–categoría.
- La **corrección de una inconsistencia real de stock**: las ventas descontaban producto.Cant_Stock pero nunca actualizaban inventario.Cant_Inventario, haciendo que ambos valores divergieran después de cada venta.

2. Requerimientos del sistema y roles

El sistema define tres roles con acceso diferenciado, controlados mediante decoradores de autorización sobre las rutas de Flask:

Rol	Acceso principal	URL de entrada	Control de acceso
Admin	Panel completo: productos, usuarios, categorías, ventas, configuración de QR de pago	/admin	@admin_requerido
Cliente	Catálogo público, carrito, checkout, historial de compras	/	@login_requerido (checkout/mis-pedidos)
Domiciliario	Panel de entregas pendientes y confirmación de entrega	/domiciliario	@domiciliario_requerido

No se contó, en esta sesión, con documentos previos de historias de usuario o casos de uso formales (la evidencia AA3-EV02 menciona 10 historias de usuario entregadas aparte). Las historias de usuario que siguen se derivaron directamente de la funcionalidad real verificada en el código:

Como...	Quiero...	Para...
Cliente	Ver el catálogo filtrando por categoría y buscando por nombre	Encontrar rápido los productos que necesito
Cliente	Agregar productos a un carrito y confirmar un pedido con dirección, método de pago y tipo de entrega	Comprar productos artesanales en línea
Cliente	Ver el historial de mis pedidos y su estado	Hacer seguimiento a mis compras

Como...	Quiero...	Para...
Admin	Crear, editar y eliminar productos, usuarios y categorías	Mantener actualizado el catálogo y el equipo
Admin	Cambiar el estado de una venta (Pendiente → Confirmado → En camino → Entregado / Cancelado)	Gestionar el ciclo de vida de cada pedido
Admin	Configurar los códigos QR de pago disponibles	Ofrecer varias cuentas/bancos para transferencias
Domiciliario	Ver los pedidos con entrega pendiente y confirmarlos	Gestionar mis entregas del día

3. Arquitectura y capas (patrón MVC)

El proyecto sigue el patrón Modelo-Vista-Controlador de forma consistente en las cuatro capas verificadas en el código real:

Capa	Ubicación	Responsabilidad
Vista	templates/*.html	Presentación con Jinja2 + JavaScript vanilla; cada plantilla es autocontenida (CSS embebido, sin framework SPA).
Rutas / API	app.py, src/routes/, src/services/	Rutas de sesión (HTML) y Blueprints REST JSON (/api/productos, /api/usuarios, /api/auth).
Controlador	src/controllers/*.py	Validaciones de negocio; retorna siempre {ok, msg[, data]}.
Modelo	src/models/*.py	Acceso a datos vía mysql-connector-python; transacciones con commit/rollback.
Configuración / BD	config/db.py + MariaDB	Conexión a la base de datos tiendalosboyacos vía XAMPP.

Diagrama de paquetes

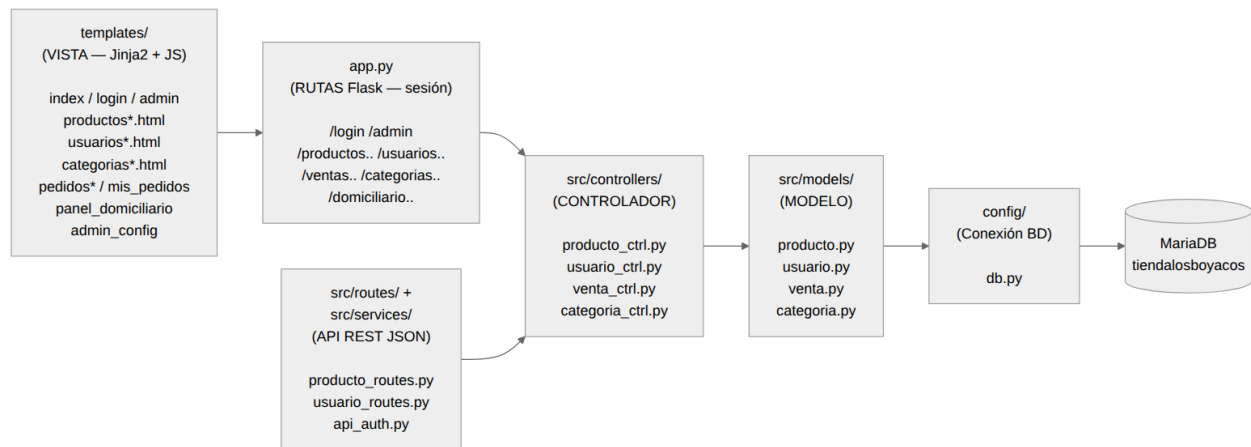


Figura 1 — Paquetes del proyecto y flujo de dependencias entre capas.

Diagrama de clases

Modelos (acceso a datos) y controladores (validaciones de negocio) de los cuatro dominios del sistema. Las líneas punteadas indican "usa"; las líneas sólidas, relaciones de la base de datos.

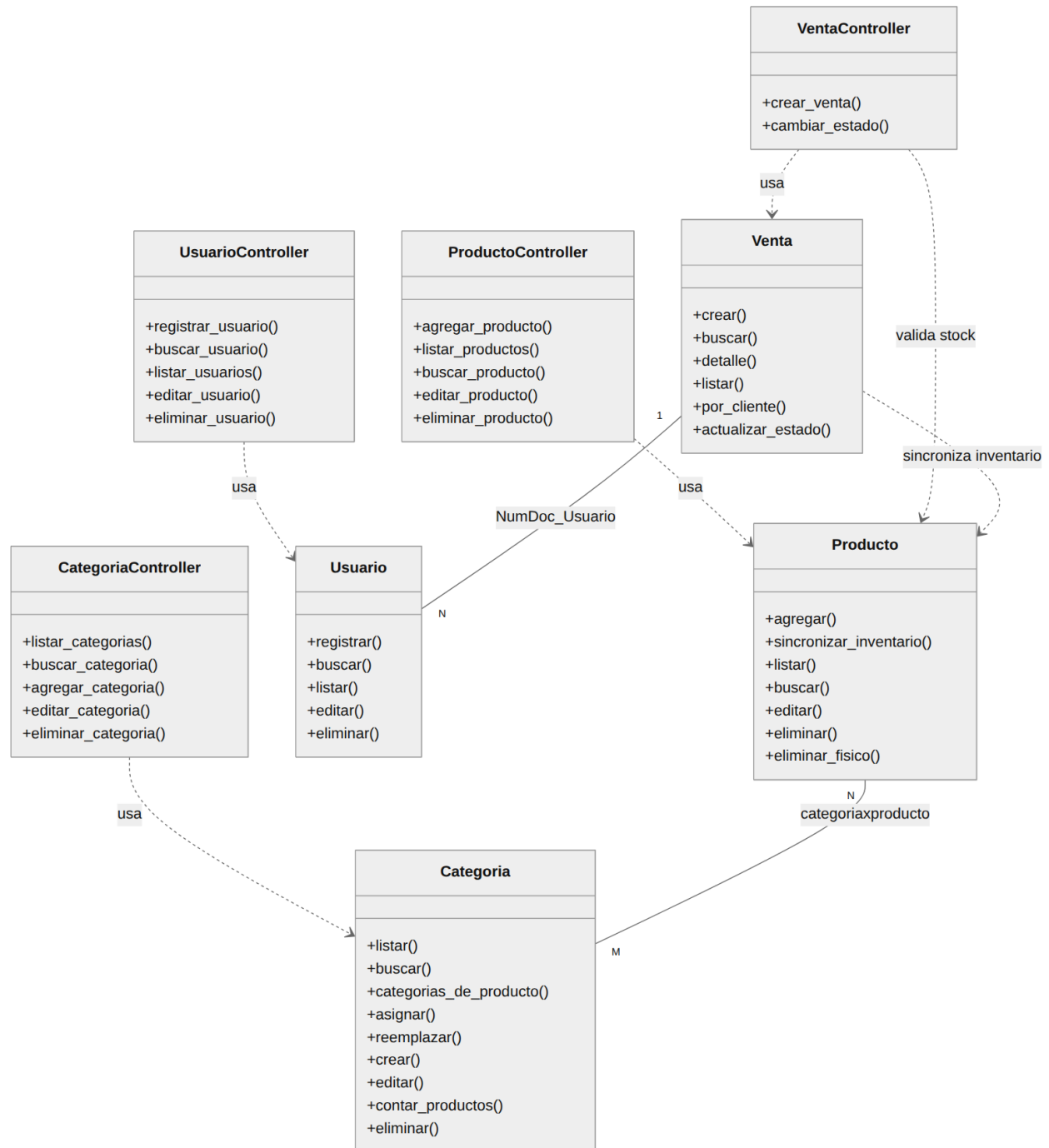


Figura 2 — Clases de modelo y controlador, y sus relaciones.

Diagrama de componentes

Componentes en tiempo de ejecución: el navegador consume tanto rutas de sesión (HTML) como los tres Blueprints REST bajo /api.

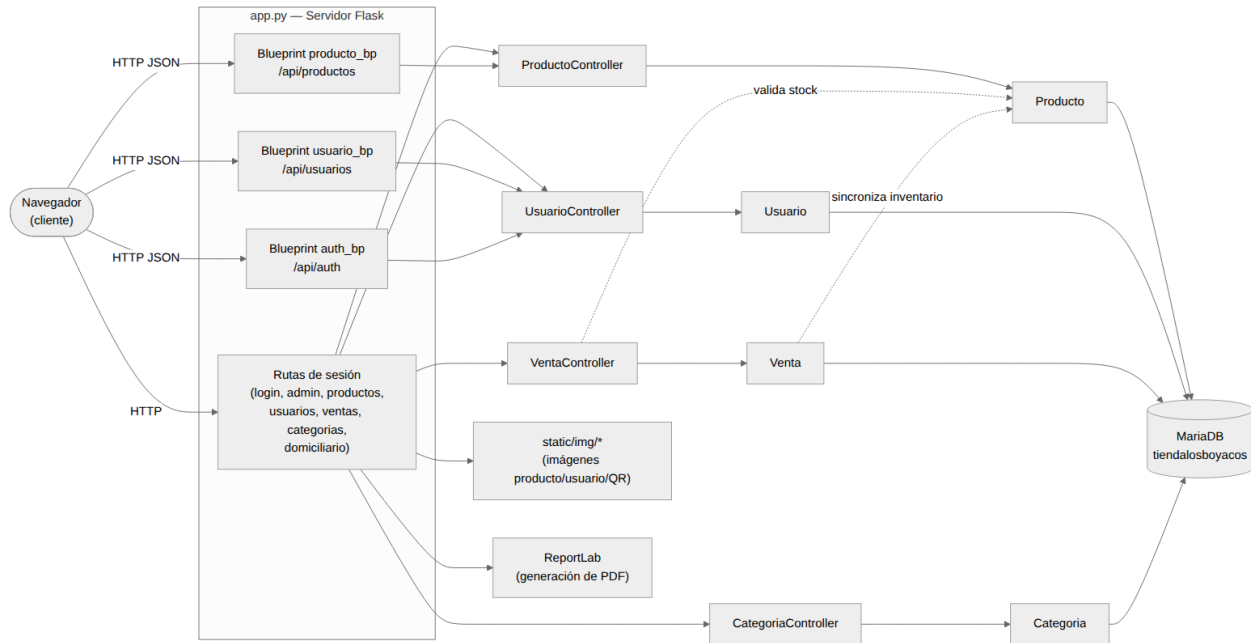


Figura 3 — Componentes del sistema en ejecución.

Mapa de navegación

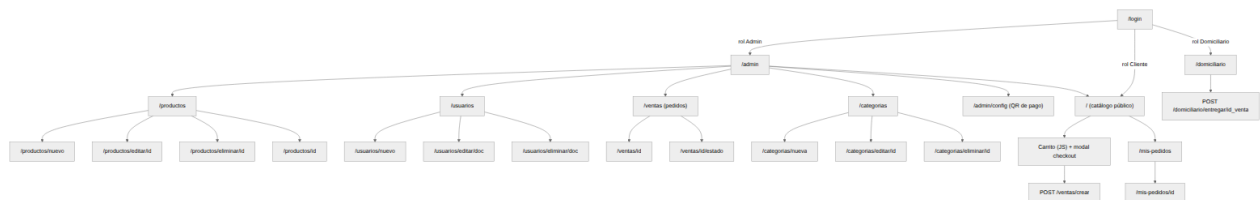


Figura 4 — Mapa de navegación por rol.

4. Mecanismos de seguridad

Mecanismo	Implementación verificada en el código
Hash de contraseñas	Werkzeug (PBKDF2-HMAC-SHA256) vía <code>generate_password_hash/check_password_hash</code> — nunca se guarda ni compara texto plano.
Control de acceso por rol	Decoradores <code>@login_requerido</code> , <code>@admin_requerido</code> , <code>@domiciliario_requerido</code> sobre cada ruta de <code>app.py</code> , basados en <code>session['usuario']['rol']</code> .
Validación de sesión	<code>app.secret_key</code> firma la cookie de sesión de Flask; sin sesión válida, toda ruta protegida redirige a <code>/login</code> .
Validación de archivos subidos	Extensión restringida a <code>png/jpg/jpeg/webp</code> y <code>MAX_CONTENT_LENGTH = 6 MB</code> por request (<code>guardar_imagen()</code> en <code>app.py</code>).
Integridad referencial	Claves foráneas con <code>ON DELETE RESTRICT/CASCADE</code> en MariaDB (p. ej. <code>categoriaxproducto</code> → <code>categoria</code>) evitan borrar registros todavía referenciados.
Borrado lógico vs. físico	Productos con ventas asociadas se desactivan (<code>Activo=0</code>) en vez de borrarse, preservando el historial de ventas.
Servicio REST de autenticación	AA5-EV01: <code>/api/auth/registro</code> y <code>/api/auth/login</code> como Blueprint independiente, stateless (JSON), reutiliza las mismas validaciones que el login por sesión.

5. Metodología de desarrollo

El proyecto se desarrolló de forma incremental, evidencia por evidencia SENA (AA1 → AA3 → AA4 → AA5), con control de versiones Git desde el primer commit. El historial real de commits (6 en total) sigue Conventional Commits (`feat:`, `fix:`, `chore:`, `db:`, `docs:`) y evidencia limpieza de código muerto en el camino (por ejemplo, el commit `929501f` elimina tablas `PEDIDO/DETALLE_PEDIDO` duplicadas del MER original).

Cuando una actividad pidió una tecnología distinta a la avalada (React), el patrón de resolución adoptado fue justificar técnicamente las equivalencias conceptuales sobre el código real, sin reescribir el proyecto — documentado en la evidencia AA4-EV03.

6. Librerías y frameworks por capa

Capa	Framework / librería	Uso
Vista	Jinja2 3.1.6	Motor de plantillas del lado del servidor
Vista	JavaScript vanilla (ES6+)	Carrito de compras, validaciones de formulario, interacción del catálogo
Rutas / API	Flask 3.1.3	Enrutamiento, sesiones, Blueprints REST
Rutas / API	Werkzeug 3.1.8	Servidor WSGI, hash de contraseñas, manejo seguro de nombres de archivo
Modelo / BD	mysql-connector-python 9.6.0	Driver de conexión a MariaDB
Reportes	ReportLab 4.4.1 + pypdf 5.4.0	Generación de comprobantes/reportes en PDF
Pruebas	pytest (>=8.0, agregado en esta actividad)	Framework de pruebas unitarias
Seguridad de dependencias	cryptography 45.0.5 + cffi 1.17.1	Dependencias transitivas de mysql-connector-python

7. Patrones de diseño aplicados

Patrón	Dónde se aplica
MVC (Modelo-Vista-Controlador)	Separación estricta entre src/models/, src/controllers/ y templates/ en las 4 capas del sistema.
Resultado uniforme / Contrato de respuesta	Todo controlador retorna {ok, msg[, data]} — un contrato consistente que la vista y la API consumen de la misma forma.
Decorator	login_requerido, admin_requerido y domiciliario_requerido envuelven vistas de Flask para aplicar control de acceso sin duplicar lógica.
Modular / Blueprint (Front Controller modular)	producto_bp, usuario_bp y auth_bp encapsulan sub-conjuntos de rutas REST reutilizando los mismos controladores que las rutas de sesión.
Static/Utility Class	Modelos y controladores exponen únicamente @staticmethod — no requieren estado de instancia, coherente con su rol de capa de acceso a datos y de validación.
Plantilla / Template Method (implícito)	El flujo agregar → sincronizar inventario → asociar categorías se repite igual en Producto.agregar y Producto.editar mediante el método compartido sincronizar_inventario().

8. Módulos codificados en esta actividad

8.1 Módulo de gestión de categorías (nuevo)

Antes de esta actividad, `src/models/categoria.py` solo permitía listar categorías y asociarlas a productos — no existía forma de crear, editar ni eliminar una categoría desde el panel de administración. Se codificó el módulo completo siguiendo exactamente las mismas convenciones ya usadas en productos y usuarios:

- **Modelo** — `Categoria.crear()`, `Categoria.editar()`, `Categoria.contar_productos()`, `Categoria.eliminar()` agregados a `src/models/categoria.py`.
- **Controlador** — nuevo `src/controllers/categoria_ctrl.py` (`CategoriaController`) con las mismas validaciones de forma (`{ok, msg[, data]}`): nombre obligatorio, máximo 30 caracteres, duplicados case-insensitive, y bloqueo de borrado si hay productos asociados.
- **Rutas** — `/categorias`, `/categorias/nueva`, `/categorias/editar/<id>`, `/categorias/eliminar/<id>` en `app.py`, protegidas con `@admin_requerido`.
- **Vistas** — `categorias.html`, `categoria_form.html` y `categoria_eliminar.html`, con el mismo lenguaje visual (colores, tipografía, tarjetas) que el resto del panel admin, más una tarjeta nueva "Categorías" en `admin.html`.

La eliminación se apoya en la integridad referencial ya existente en la base de datos: la FK `fk_categoriaproducto_categoria` tiene `ON DELETE RESTRICT`, así que el controlador primero cuenta los productos asociados (`Categoria.contar_productos()`) y solo intenta el `DELETE` cuando el conteo es cero — evitando depender únicamente de que la base de datos rechace la operación.

8.2 Corrección: inventario no se sincronizaba tras una venta

Se verificó código real que confirmaba el pendiente ya señalado en `migraciones/2026_08_09_fix_esquema.sql` (sección 8, redundancia `Cant_Stock/Inventario`):

`Venta.crear()` descontaba `producto.Cant_Stock`, pero nunca actualizaba

`inventario.Cant_Inventario`. Alta y edición de producto sí sincronizaban ambos valores — solo la venta los dejaba divergir.

Solución aplicada (Opción A de la propia migración, adaptada a lo que es verificable sin acceso en vivo a la base de datos desde este entorno): se extrajo la lógica de sincronización a un método reutilizable y se invocó también desde la venta.

`src/models/producto.py` — método compartido

```
@staticmethod
def sincronizar_inventario(cursor, id_producto, cant_stock, observacion):
    cursor.execute("""
        INSERT INTO inventario
            (Id_Producto, Cant_Inventario, Fech_Registro, Observacion)
        VALUES (%s, %s, NOW(), %s)
        ON DUPLICATE KEY UPDATE
            Cant_Inventario = VALUES(Cant_Inventario),
            Fech_Registro = NOW(),
            Observacion = VALUES(Observacion)
    """, (id_producto, cant_stock, observacion))
```

src/models/venta.py — se invoca tras descontar stock

```
cursor.execute("SELECT Cant_Stock FROM producto WHERE Id_Producto = %s", (item["id_producto"],))
nuevo_stock = cursor.fetchone()[0]
Producto.sincronizar_inventario(cursor, item["id_producto"], nuevo_stock, f"Salida por venta
#{id_venta}")
```

Con esto, alta, edición y venta comparten el mismo mecanismo de sincronización — cerrando la divergencia sin necesitar una migración de esquema que no podía validarse en vivo desde este entorno (sin acceso de red al MariaDB del equipo del estudiante). Queda documentado como mejora futura convertir inventario en una bitácora histórica real (múltiples filas por producto) tal como sugiere la Opción A original de la migración.

9. Pruebas unitarias

El archivo previo `tests/test_modelos.py` llamaba a `Usuario.registrar(...)` con argumentos posicionales incompletos (faltaban `indicativo` y `password`, ambos obligatorios) y solo imprimía resultados sin ningún `assert` — no era ejecutable como prueba automatizada. Se reemplazó por una suite `pytest` real, aislada de la base de datos mediante `mocks` de `get_connection()/get_cursor()`, cubriendo los 4 módulos del sistema:

Archivo	Qué cubre
<code>tests/test_modelos.py</code>	Modelo Producto: alta con sincronización de inventario, edición, borrado lógico, rollback ante error de BD
<code>tests/test_producto_ctrl.py</code>	ProductoController: validaciones de precio/stock, borrado lógico automático cuando hay ventas asociadas
<code>tests/test_usuario.py</code>	UsuarioController: hash de contraseña, contraseña mínima, documento duplicado, confirmación de contraseña
<code>tests/test_venta.py</code>	Modelo Venta y VentaController: la corrección de sincronización de inventario, stock insuficiente, carrito vacío, estado inválido
<code>tests/test_categoria.py</code>	Modelo y controlador de Categoría (módulo nuevo): nombre obligatorio, duplicados, bloqueo de borrado con productos asociados, error de FK

Ejecución real de la suite completa (`pytest -q`), en un entorno con las dependencias de `requirements.txt` instaladas:

```
$ pytest -q
..... [100%]
28 passed in 1.97s
```

Se agregó `pytest>=8.0` a `requirements.txt` bajo una nueva sección "Pruebas unitarias". Además, se validó que `app.py` sigue importando sin errores y que las 4 rutas nuevas de categorías quedan registradas correctamente en el mapa de rutas de Flask tras los cambios.

10. Configuración de entornos de desarrollo y pruebas

Elemento	Configuración
Sistema operativo	Windows 10/11 (equipo del estudiante)
Servidor local	XAMPP 8.x (Apache + MariaDB)
Ruta del proyecto	C:\xampp\htdocs\PROYECTOS\tiendalosboyacos-python
Lenguaje	Python 3.13
Base de datos	MariaDB — esquema tiendalosboyacos, conexión root sin contraseña vía config/db.py
Instalación de dependencias	pip install -r requirements.txt
Arranque de la aplicación	python app.py → http://localhost:5000
Ejecución de pruebas	pytest -q, desde la raíz del proyecto (tests/conftest.py agrega la raíz al path para que "from src.models..." resuelva igual que en app.py)
Entorno de verificación de esta evidencia	VM Linux con acceso de archivos al proyecto real; sin conectividad de red hacia el MariaDB de XAMPP del estudiante — las pruebas unitarias se diseñaron para no depender de una conexión real a la base de datos

11. Repositorio de control de versiones

Git 2.49.0, repositorio en GitHub: github.com/MARINOPEREZ/tiendalosboyacos-python (rama main). Convención de commits: Conventional Commits (feat:, fix:, chore:, db:, docs:). Historial verificado en esta sesión (6 commits): estructura base → normalización/migración de esquema → panel domiciliario → limpieza de código muerto de PEDIDO → servicio REST de autenticación (AA5-EV01) → fix de codificación UTF-8 en Windows.

12. Conclusiones y entregables

El sistema cubre de forma funcional los tres roles definidos (Admin, Cliente, Domiciliario) sobre una arquitectura MVC consistente, con servicios REST adicionales para consumo externo. Esta actividad cerró la brecha de un módulo administrativo faltante (categorías) y corrigió una inconsistencia real de datos (sincronización de inventario en ventas), además de reemplazar pruebas manuales no ejecutables por una suite pytest real y en verde.

Entregables de esta evidencia:

- Código fuente actualizado en el repositorio del proyecto (carpeta local y GitHub).
- Este documento PDF, con diagramas de clases, paquetes, componentes y mapa de navegación generados a partir del código real.
- Suite de pruebas unitarias (tests/) ejecutable con `pytest -q`.
- URL del repositorio: github.com/MARINOPEREZ/tiendalosboyacos-python

Pendiente sugerido para una futura evidencia: convertir inventario en una bitácora histórica real (múltiples movimientos por producto, no un valor único sobrescrito) y, si se retoma la evidencia AA5-EV03, decidir si sus 4 Blueprints adicionales (src/api/*) se integran a app.py o se mantienen como evidencia aparte.